Deconstructing Taxi-V3 with Maximum Entropy Inverse Reinforcement Learning

Peter Adeyemo
Carnegie Mellon University - Africa
padeyemo@andrew.cmu.edu

Dev Rawal
Carnegie Mellon University - Africa
dprawal@andrew.cmu.edu

1 Introduction

Inverse Reinforcement Learning (IRL) addresses the challenge of inferring an agent's underlying reward function by observing its behavior. Unlike standard reinforcement learning, where the reward function is known, IRL seeks to find a reward function that makes the expert's observed behavior appear near-optimal. This is a powerful paradigm for domains where reward functions are complex and difficult to specify by hand, such as autonomous driving or robotic manipulation, as it allows for learning from human demonstration.

A fundamental challenge in IRL is ambiguity; many different reward functions can explain the same expert policy. The Maximum Entropy (MaxEnt) IRL framework [1], provides a principled solution to this problem. It operates on the principle of maximum entropy, selecting the reward function that makes the expert's behavior optimal while remaining maximally uncertain about all other possible behaviors. This results in a stochastic policy in which the probability of a trajectory is exponentially proportional to its reward.

In this project, we used the Taxi-V3 environment from the Gymnasium library [6] as a tractable test bed (500 states) to analyze the stability and robustness of MaxEnt IRL. While prior work has established the theoretical foundations of MaxEnt, practical implementations often face challenges regarding algorithmic stability and sensitivity to demonstration quality.

Our contributions in this paper are twofold:

1. We provide a verified implementation of the MaxEnt IRL algorithm on the discrete Taxi-V3 domain adapted from previous implementations[9] and applied and employed the L2 regularization for algorithmic stability.
2. We empirically evaluate the robustness of the algorithm by training it on suboptimal experts. By injecting stochastic noise into the expert demonstrations, we test the hypothesis that MaxEnt's probabilistic nature allows it to filter out inconsistent errors and recover the true task objective.

2 Related Work

Classical Inverse Reinforcement Learning (IRL) methods aim to recover a reward function that explains expert demonstrations. One of the earliest formulations, Apprenticeship Learning [4], proposes matching expert feature expectations to obtain near-optimal behavior. While influential, this approach suffers from ambiguity due to the existence of multiple reward functions compatible with the same expert behavior.

The Maximum Entropy IRL (MaxEnt IRL) framework introduced by Ziebart et al. [1] addresses this ambiguity by selecting the reward that induces a Maximum Entropy policy consistent with expert demonstrations. This formulation yields a unique probabilistic model of expert behavior and has become a foundational method for feature-based IRL. Theoretically, this probabilistic perspective implies an inherent robustness to suboptimal experts. As investigated by recent work on noisy demonstrations [12], MaxEnt IRL should be able to filter out "random action noise" because the

model assumes that agents act stochastically proportional to reward. However, in practice, MaxEnt IRL is known to be computationally sensitive: gradient updates can be unstable, feature expectations may deviate significantly, and convergence often depends critically on the choice of learning rate.

Efforts to improve stability and robustness in IRL are an active area of research. Boularias et al. [5] propose the Relative Entropy IRL, which uses importance sampling to reduce variance in gradient estimation. More recently, Chen et al. [13] proposed "Universal Stabilization" techniques (MeSO) to mitigate the "spiky" Q-function updates inherent in maximum entropy optimization, highlighting that algorithmic stability remains a critical open challenge even in 2025.

This project builds on these analytical perspectives directly. Rather than implementing complex stabilization objectives such as MeSO [13], we focus on quantifying the performance of the original algorithm under specific constraints. We specifically investigate two interventions: (1) empirically testing the algorithm's robustness to suboptimal experts by introducing ϵ-greedy noise as suggested by [12], and (2) implementing L2 regularization to prevent weight divergence. Our work analyzes how these adjustments influence reward recovery in the Taxi-V3 environment.

3 Dataset Description & Exploration

3.1 Environment: Taxi-V3

The data set is a collection of expert trajectories generated in the `Taxi-V3` environment. The environment has the following characteristics:

- State Space: A discrete, finite state space of 500 states. Each state is a unique integer that encodes four pieces of information:
  - Taxi Row (0-4)
  - Taxi Column (0-4)
  - Passenger Location (0-4, where 0-3 are R,G,Y,B and 4 is "in taxi")
  - Destination (0-3, corresponding to R,G,Y,B)

  The state is encoded as:

  $$s = ((\text{taxi}_{\text{row}} \cdot 5 + \text{taxi}_{\text{col}}) \cdot 5 + \text{passenger}_{\text{loc}}) \cdot 4 + \text{destination}_{\text{idx}}$$

- Action Space: A discrete action space of 6 actions:
  - 0: Move South
  - 1: Move North
  - 2: Move East
  - 3: Move West
  - 4: Pickup Passenger
  - 5: Drop-off Passenger

3.2 Trajectory Dataset

The data set for the IRL algorithm, as mentioned earlier, is a collection of 200 expert trajectories. Each trajectory is a sequence of (state, action) tuples $\tau = \{(s_0, a_0), (s_1, a_1), ..., (s_t, a_t)\}$, generated by the synthetic expert. The synthetic experts were obtained through Value Iteration in the environment. This data set represents the "expert demonstration" that the MaxEnt IRL algorithm will attempt to mimic.

4 Baseline Methodology

Our baseline methodology relies on a shared feature representation to bridge the gap between the expert's behavior and the learner's reward recovery. We first define this feature space and then proceed in two phases: (1) generating ground-truth demonstrations via Value Iteration, and (2) recovering the reward function using the Maximum Entropy IRL algorithm.

4.1 Feature Engineering

We defined a feature mapping $\phi(s, a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^d$ to represent the dynamics of the environment. For this baseline implementation, we utilized a 5-dimensional feature vector ($d = 5$) designed to capture the essential sub-goals of the taxi task:

1. **Successful Dropoff (Binary):** Activates when the agent successfully executes the "Dropoff" action at the correct destination.
2. **Successful Pickup (Binary):** Activates when the agent successfully executes the "Pickup" action at the passenger's location.
3. **Negative Distance to Passenger:** The negative Manhattan distance between the taxi and the passenger. This serves as a dense reward signal to guide the agent toward the pickup location.
4. **Negative Distance to Destination:** The negative Manhattan distance between the taxi and the destination (active only after pickup). This guides the agent to the drop-off zone.
5. **Step Cost:** A constant feature (always 1.0) added to every state-action pair to penalize longer trajectories and encourage efficiency.

The reward function is then modeled as $R(s, a) = w^T \phi(s, a)$, where $w \in \mathbb{R}^5$ is the weight vector learned by the MaxEnt algorithm.

4.2 Phase 1: Expert Policy Generation

To create a synthetic expert, we define a ground-truth reward function as a linear combination of features. We utilized Value Iteration to solve the Bellman Optimality Equation for the Taxi-V3 environment:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V^*(s')]$$

Using the converged value function V^*, we derived a deterministic optimal policy π^*. We then implemented this policy to generate a dataset of $N = 200$ expert trajectories, denoted as $\mathcal{D} = \{\tau_1, \tau_2, \ldots, \tau_N\}$.

4.3 Phase 2: Maximum Entropy IRL

We implemented the MaxEnt IRL algorithm [1], which models the probability of a trajectory τ as proportional to the exponent of its reward:

$$P(\tau|w) = \frac{1}{Z(w)} \exp\left(\sum_t w^T \phi(s_t)\right)$$

The algorithm iteratively optimizes the weights w to maximize the likelihood of the expert demonstrations. This is equivalent to minimizing the difference between the expert's feature expectations and the learner's feature expectations. The gradient update rule is given by:

$$\nabla L(w) = \tilde{\phi}_{\text{expert}} - \sum_\tau P(\tau|w)\phi(\tau)$$

We approximated the learner's expectations using the forward-backward algorithm on the trellis of states, allowing for efficient computation of state visitation frequencies without explicitly enumerating all paths.

5 Improved Methodology

Building on the baseline implementation, we investigate two critical aspects of Inverse Reinforcement Learning in practical settings: (1) the ability to learn from imperfect, suboptimal demonstrators, and (2) the algorithmic stability of the learning process.

5.1 Modeling Suboptimal Experts

The standard MaxEnt IRL assumes that the expert is optimally maximizing a reward function. However, real-world demonstrations often contain errors, lapses in judgment, or stochastic behavior. To evaluate the robustness of MaxEnt IRL to such imperfections, we introduce a stochastic "Suboptimal Expert" model.

Instead of strictly following the optimal policy π^* derived in Phase 1, we generate trajectories using a ϵ-greedy policy. At each time step, the expert chooses a random action with probability ϵ, and the optimal action with probability $1 - \epsilon$. The probability of taking action a in state s is given by:

$$\pi_{\text{noisy}}(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \pi^*(s) \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

We generated a new datasets with noise levels $\epsilon = 0.2$ to simulate expert degradation. This experimental design is inspired by recent investigations into noisy demonstrations [12], testing the hypothesis that MaxEnt's probabilistic formulation can filter out uncorrelated noise while retaining the underlying task signal.

5.2 Algorithmic Stability via L2 Regularization

As noted in related work [3, 13], the standard MaxEnt objective function is under-constrained, often leading to unbounded growth in weight magnitudes. This weight explosion causes the resulting policy to become over-confident and deterministic, effectively overfitting to the specific finite set of expert trajectories.

To mitigate this, we introduce L2 Regularization (weight decay) into the optimization objective. We modify the log-likelihood objective function $L(w)$ by adding a penalty term proportional to the squared magnitude of the weights:

$$J(w) = L(w) - \frac{\lambda}{2}||w||^2$$

where λ is the regularization coefficient. This modification alters the gradient ascent update rule. The new gradient, $\nabla J(w)$, is calculated as:

$$\nabla J(w) = \underbrace{\left(\tilde{\phi}_{\text{expert}} - \sum_{\tau} P(\tau|w)\phi(\tau)\right)}_{\text{Standard MaxEnt Gradient}} - \underbrace{\lambda w}_{\text{L2 Penalty}}$$

We investigate the impact of this regularization on both the convergence stability (measured by weight variance) and the final performance of the policy, hypothesizing that $\lambda > 0$ will prevent divergence and produce more robust policies.

6 Results

In this section, we evaluate the performance of the MaxEnt IRL algorithm using two primary metrics:

- **Success Rate:** The percentage of test episodes (out of 50) in which the agent successfully delivered the passenger.
- **Policy Disagreement:** The number of states (out of 500) where the learned policy differs from the ground-truth optimal policy.

We compared three experimental configurations: the **Baseline** (unregularized training on perfect data), **L2 Regularized** (regularized training on perfect data), and **Suboptimal Expert** (unregularized training on 20% noisy data).

Table 1 summarizes the performance across these three methods.

The Baseline method performed poorly, achieving only a 48% success rate. In 26 episodes, the agent failed to converge to the goal, often entering infinite loops that resulted in a timeout penalty. This

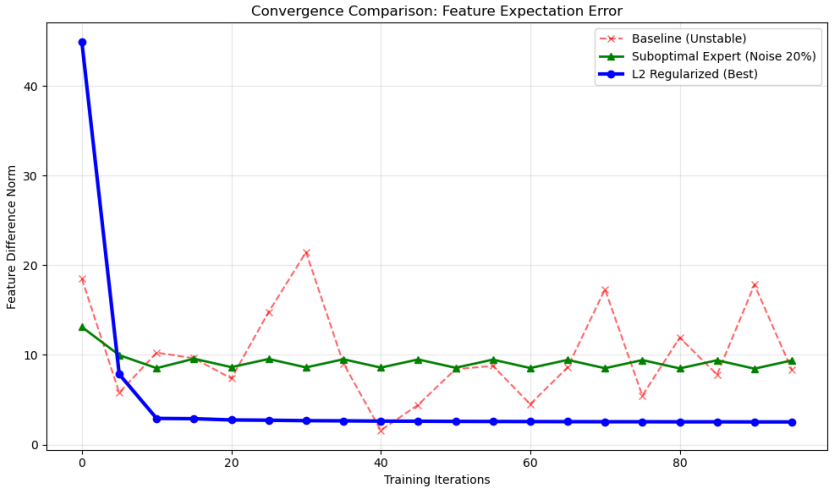

Figure 1: **Convergence Comparison.**

Method	Success Rate	Failures	Policy Diff	Convergence
Baseline (Unregularized)	48%	26/50	74 states	Unstable
L2 Regularized	**92%**	**4/50**	**32 states**	**Stable**
Suboptimal Expert (Noise 20%)	82%	9/50	68 states	Stable (High Error)

Table 1: Performance comparison over 50 test episodes.

correlates with the high policy disagreement (74 states), indicating the learner failed to generalize from the expert trajectories.

The L2 Regularized method ($\lambda = 0.05$) achieved the best performance, with a 92% success rate and the lowest policy disagreement (32 states). The regularization effectively smoothed the optimization landscape, preventing the overfitting observed in the baseline.

The Suboptimal Expert method, despite being trained on imperfect data ($\epsilon = 0.2$), achieved a surprisingly robust 82% success rate. While it did not match the precision of the L2 model, it significantly outperformed the Baseline, suggesting that the introduction of noise may have inadvertently acted as a regularizer, preventing the agent from overfitting to specific expert paths.

7 Discussion and Analysis

This project highlighted several critical insights regarding the stability and robustness of Inverse Reinforcement Learning.

7.1 The Necessity of Regularization

Our most significant finding is that standard MaxEnt IRL is inherently unstable on the Taxi-V3 domain without regularization. The Baseline model's poor performance (48% success) demonstrates that "perfect" expert data is insufficient if the learning algorithm is under-constrained. The optimization process tends to drift, learning weights that explain the specific demonstrations perfectly but fail to generalize to the rest of the state space. L2 regularization solved this by penalizing large weights, forcing the agent to learn a simpler, more robust reward function.

7.2 Robustness to Noise

We observed that MaxEnt IRL is highly robust to suboptimal demonstrations. Even when the expert made random errors 20% of the time, the learner successfully recovered a viable policy (82% success). Analysis of the recovered weights reveals the agent adopted a risk-averse strategy, learning a high

penalty for time steps. This suggests that in the presence of noise, MaxEnt naturally filters out inconsistent random actions and locks onto the consistent signal, which in this case, was the urgency to reach the destination.

7.3 Reward Ambiguity

We observed that different weight vectors produced similar behaviors. The Suboptimal Expert learned weights that were numerically very different from the Ground Truth (e.g., a normalized step cost of -0.38 vs -0.05), yet the resulting policy was highly effective. This confirms the theoretical challenge of reward ambiguity in IRL: the algorithm recovers *a* reward function that explains behavior, not necessarily the *true* reward function intended by the designer.

8 Division of Work

The project work was distributed to leverage individual strengths while ensuring both members gained a deep understanding of the core concepts. Both team members initially explored the Taxi-V3 environment and conducted a literature review of the foundational MaxEnt IRL paper by Ziebart et al. [1].

Specific contributions were divided as follows:

- **Dev** led the implementation of the synthetic expert (Phase 1) and the feature engineering pipeline. He was responsible for refining the core MaxEnt algorithm for the final submission and implementing the L2 Regularization stability improvement. He also performed the final code refactoring and cleaning.
- **Peter** established the initial MaxEnt IRL baseline by adapting open-source implementations to the Taxi-V3 domain. He led the investigation into robustness against Suboptimal Experts. Additionally, Peter was responsible for experimental execution, compiling the test results, and leading the interpretation of the data for the final report.

Both team members collaborated closely on debugging the combined codebase and co-authored the final report, synthesizing the results into the discussion and analysis sections.

References

[1] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey, "Maximum Entropy Inverse Reinforcement Learning," Proc. AAAI, 2008.

[2] A. J. Snoswell, S. P. N. Singh, and N. Ye, "Revisiting Maximum Entropy Inverse Reinforcement Learning: New Perspectives and Algorithm," in 2020 Ieee Symposium Series on Computational Intelligence (ssci), New York: IEEE, 2020, pp. 241–249. Accessed: Nov. 06, 2025. [Online]. Available: `https://arxiv.org/pdf/2012.00889`

[3] L. Song, Q. Guo, I. A. Channa, and Z. Wang, "A Survey of Maximum Entropy-Based Inverse Reinforcement Learning: Methods and Applications," Symmetry-Basel, vol. 17, no. 10, p. 1632, Oct. 2025, doi: 10.3390/sym17101632.

[4] P. Abbeel and A. Y. Ng, "Apprenticeship learning via inverse reinforcement learning," in Proceedings of the twenty-first international conference on Machine learning, in ICML '04. New York, NY, USA: Association for Computing Machinery, July 2004, p. 1. doi: 10.1145/1015330.1015430.

[5] A. Boularias, J. Kober, and J. Peters, "Relative Entropy Inverse Reinforcement Learning," in Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, JMLR Workshop and Conference Proceedings, June 2011, pp. 182–189. Accessed: Nov. 07, 2025. [Online]. Available: `https://proceedings.mlr.press/v15/boularias11a.html`

[6] M. Towers et al., "Gymnasium: An API for Reinforcement Learning," 2023. [Online]. Available: `https://gymnasium.farama.org/`

[7] Farama Foundation (2022). Taxi-V3. [Online]. Gymnasium Documentation. Available: `https://gymnasium.farama.org/environments/toy_text/taxi/`

[8] Finn, C. (2020). Inverse Reinforcement Learning. CS285 Bootcamp Slides, Stanford University. [Online]. Available: `https://ai.stanford.edu/~cbfinn/_files/bootcamp_inverserl.pdf`

[9] qzed (2016). irl-maxent: Implementation of Maximum Entropy Inverse Reinforcement Learning. [GitHub]. Available: https://github.com/qzed/irl-maxent

[10] Sehgal, K. (2020). Reinforcement Learning: An Easy Introduction to Value Iteration. [Online]. Towards Data Science. Available: https://towardsdatascience.com/reinforcement-learning-an-easy-introduction-to-value-iteration-e4cfe0731fd5

[11] Taresin, M. (2017). Simple gridworld Value Iteration. [Gist]. Available: https://gist.github.com/Taresin/a090274fbaf092ad649e4e32e22ecaf4

[12] M. Mitra, G. Kumar, P. Chakrabarti, and P. Biswas, "Investigating Inverse Reinforcement Learning during Rapid Aiming Movement in Extended Reality and Human-Robot Interaction," J. Hum.-Robot Interact., vol. 14, no. 4, p. 68:1-68:33, July 2025, doi: 10.1145/3736423.

[13] X. Chen et al., "Universal Stabilization for Maximum Entropy Optimization in Reinforcement Learning," IEEE Trans Neural Netw Learn Syst, vol. PP, Nov. 2025, doi: 10.1109/TNNLS.2025.3626050.